



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

Métodos

Professor: Henrique Delegrego

Métodos

Introdução

- Blocos de código que executam uma tarefa específica
- Quando um método é chamado, o programa executa o código dentro dele
- Permitem decompor o programa em partes menores, **reutilizáveis** e **modularizadas**
- Serão executados quando chamados pelo método **main**
- Usa o mesmo tipo de nomenclatura de variáveis (**camelCase**)
- Por representarem ações ou comportamentos, os nomes devem ser **expressões verbais**, geralmente no **infinitivo**

Métodos

Exemplo de código

```
public class CodigoSemMetodo {  
    // Exemplo de código sem uso de métodos  
    public static void main(String[] args) {  
        System.out.println("Olá, meu nome é Henrique");  
        System.out.println("Eu sou professor de desenvolvimento de sistemas");  
    }  
}
```

Métodos

Exemplo de código

```
public class Metodo {  
  
    public static void main(String[] args) {  
  
        // Chamada do método  
        apresentarProfessor();  
  
    }  
  
    // Criação do método  
    public static void apresentarProfessor() {  
        System.out.println("Olá, meu nome é Henrique");  
        System.out.println("Eu sou professor de desenvolvimento de sistemas");  
    }  
  
}
```

Agora:

- Vamos ver como métodos são feitos
- Fazer exercício 1 e 2

Métodos

Métodos com parâmetros

- **Parâmetros** são variáveis definidas na declaração do método para receber valores de entrada
- Quando um método possui parâmetros, significa que ele pode receber dados externos e usar esses dados em sua lógica
- Torna os métodos mais flexíveis e reutilizáveis, pois, em vez de codificar valores fixos, você pode passar entradas diferentes a cada invocação do método

Métodos

Exemplo de código

```
5 public class CodigoSemMetodo {  
6  
7     // Exemplo de código sem utilizar métodos  
8     public static void main(String[] args) {  
9         Scanner input = new Scanner(System.in);  
10  
11         System.out.println("Insira um número");  
12         int numero = input.nextInt();  
13  
14         int aoQuadrado = numero * numero;  
15  
16         System.out.println(aoQuadrado);  
17     }  
18  
19 }
```

Métodos

Exemplo de código

```
7 public class ExemploMetodoComParametro {
8
9     // Este é um MÉTODO que recebe um PARÂMETRO.
10    // "int num" significa que o método espera receber um número int quando chamado
11    public static void aoQuadrado(int num) {
12        // Aqui usamos o valor recebido (num) e calculamos o quadrado dele
13        int resultado = num * num;
14        System.out.println(resultado);
15    }
16
17    public static void main(String[] args) {
18
19        Scanner input = new Scanner(System.in);
20
21        System.out.println("Insira um número");
22        int numero = input.nextInt();
23
24        // Chamamos o método "aoQuadrado" e passamos "numero" como argumento
25        // Esse valor será recebido pelo parâmetro "num" no método
26        aoQuadrado(numero);
27    }
28 }
```


Métodos

Exemplo de código

```
7 public class ExemploMetodoComParametro {
8
9     // (int num) é o PARÂMETRO
10    // O método recebe um valor inteiro quando o método é chamado
11    public static void numeroParOuImpar(int num) {
12
13        if (num % 2 == 0) {
14            System.out.println("Par");
15        } else {
16            System.out.println("Ímpar");
17        }
18    }
19
20    public static void main(String[] args) {
21
22        Scanner input = new Scanner(System.in);
23
24        System.out.println("Entre com um número");
25        int numero = input.nextInt();
26
27        // Chamamos o método e passamos o valor da variável "numero" como argumento
28        // Esse valor vai ser recebido no parâmetro "num" do método
29        numeroParOuImpar(numero);
30    }
31 }
```

Métodos

Exemplo de código

```
7 public class ExemploMetodoComParametro {
8
9     // Este método recebe DOIS parâmetros: numerador e denominador
10    // IMPORTANTE: a ordem importa! Primeiro numerador, depois denominador
11    public static void dividirNumeros(double numerador, double denominador) {
12        System.out.println(numerador / denominador);
13    }
14
15    public static void main(String[] args) {
16
17        Scanner input = new Scanner(System.in);
18
19        System.out.println("Insira o numerador");
20        double numerador = input.nextDouble();
21
22        System.out.println("Insira o denominador");
23        double denominador = input.nextDouble();
24
25        // Chamamos o método dividirNumeros
26        // ATENÇÃO: os valores devem ser passados na mesma ordem em que os parâmetros foram definidos no método
27        // 1º numerador, 2º denominador
28        dividirNumeros(numerador, denominador);
29
30        // Exemplo do que NÃO fazer:
31        // dividirNumeros(denominador, numerador);
32        // Isso inverteria os valores e mudaria completamente o resultado!
33    }
34 }
```

Métodos

Exemplo de código

```
5 public class ExemploMetodoComParametro {
6
7     public static void main(String[] args) {
8
9         int vetor[] = { 1, 2, 3, 4, 5 };
10
11         // Chamamos o método passando o vetor inteiro como argumento.
12         // IMPORTANTE: estamos passando um int[] (vetor), e não um int simples.
13         imprimirVetor(vetor);
14     }
15
16     // Este método recebe como parâmetro um VETOR de inteiros, não um único número
17     // Ou seja:
18     // ✓ correto: int vetor[]
19     // x errado: int vetor (isso seria apenas um número, não um vetor)
20     public static void imprimirVetor(int vetor[]) {
21         for (int i = 0; i < vetor.length; i++) {
22             System.out.print(vetor[i] + " ");
23         }
24     }
25 }
```

Agora:

- Vamos ver como os métodos com parâmetros são feitos
- Fazer exercício 3 a 6

Métodos

Métodos com retorno

- Método que produzem e retornam um valor para quem os chamou, guiadas por um tipo de retorno declarado e garantido pela instrução **return**
- Ao contrário dos métodos **void**, que apenas executam instruções, os métodos com **return** são definidos com um tipo de retorno específico, declarando explicitamente que tipo de dado o método produzirá após a execução
- Quando o Java encontra um **return**, ela encerra imediatamente o método e obrigatoriamente retorna o valor especificado para onde o método foi invocado
- Esse valor será a única informação que será externalizada pelo método

Métodos

Exemplo de código

```
4 public class ExemploMetodoComRetorno {
5
6     // Método que CALCULA e RETORNA um valor double
7     public static double obterArea(double base, double altura) {
8
9         double resultado = base * altura;
10
11         // "return" devolve o valor calculado para quem chamou o método
12         return resultado;
13     }
14
15     public static void main(String[] args) {
16
17         double base = 10;
18         double altura = 5;
19
20         // Chamamos o método e guardamos o valor retornado na variável "area"
21         double area = obterArea(base, altura);
22
23         // Imprimimos o valor que foi retornado
24         System.out.println(area);
25
26         // Outra forma: Chamar o método diretamente dentro do println
27         // O valor retornado já é impresso sem precisar de variável
28         System.out.println(obterArea(base, altura));
29     }
30 }
```

Métodos

Exemplo de código

```
5 public class ExemploMetodoComRetorno {
6
7     /**
8      * Realiza a soma de dois números inteiros.
9      *
10     * @param num1 o primeiro número a ser somado
11     * @param num2 o segundo número a ser somado
12     * @return o resultado da soma entre num1 e num2
13     */
14     public static int somarNumero(int num1, int num2) {
15         return num1 + num2;
16     }
17
18     public static void main(String[] args) {
19
20         // Como o método retorna um valor, podemos armazenar esse valor em uma variável
21         int soma = somarNumero(5, 2);
22
23         // E Imprimir o valor armazenado na variável 'soma'
24         System.out.println(soma);
25
26         // Outra forma de usar um método com retorno:
27         // Em vez de guardar o valor em uma variável, podemos usar o retorno diretamente
28         System.out.println(somarNumero(5, 2));
29     }
30
31 }
```

Métodos

Exemplo de código

```
5 public class ExemploMetodoComRetorno {
6
7     /**
8      * Retorna o nome completo a partir do nome e sobrenome informados.
9      *
10     * @param nome      o primeiro nome da pessoa
11     * @param sobrenome o sobrenome da pessoa
12     * @return uma String contendo o nome completo (nome + espaço + sobrenome)
13     */
14     public static String obterNomeCompleto(String nome, String sobrenome) {
15         return nome + " " + sobrenome;
16     }
17
18     public static void main(String[] args) {
19
20         String nome = obterNomeCompleto("Henrique", "Delegregio");
21         System.out.println(nome);
22
23         // Ou
24         System.out.println(obterNomeCompleto("Henrique", "Delegregio"));
25
26     }
27
28 }
```

Agora:

- Vamos ver como os métodos com retorno são feitos
- Terminar a lista de exercício



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC